

C++ 프로그래밍

포인터 (Pointer) — C와 C++의 차이

C 포인터 복습

핵심 연산자

```
int x = 42;

int* p = &x;    // & : 주소 연산자 (x의 주소)
*p = 100;       // * : 역참조 연산자 (p가 가리키는 값)

printf("%d\n", x);    // 100
printf("%p\n", p);    // 0x7fff... (주소)
printf("%d\n", *p);   // 100
```

포인터와 배열

```
int arr[3] = {10, 20, 30};
int* p = arr;    // 배열 이름 = 첫 번째 원소의 주소

printf("%d\n", *(p + 0)); // 10
printf("%d\n", *(p + 1)); // 20
printf("%d\n", p[2]);     // 30 (배열 표기와 동일)
```

C 동적 할당

```
#include <stdlib.h>

// 단일 변수
int* p = (int*)malloc(sizeof(int));
*p = 42;
free(p);
p = NULL;

// 배열
int n = 5;
int* arr = (int*)malloc(n * sizeof(int));
if (arr == NULL) { /* 실패 처리 */ }

for (int i = 0; i < n; i++)
    arr[i] = i * 10;

free(arr);
arr = NULL;
```

C에서는 malloc / free, 타입 캐스팅, NULL 체크가 항상 함께 따라온다.

nullptr — C++의 null 포인터

C++11부터 `NULL` 대신 `nullptr` 을 사용한다.

NULL의 문제점

```
// C에서 NULL은 단순히 정수 0 또는 (void*)0
#define NULL 0

void func(int n) { printf("int\n"); }
void func(int* p) { printf("ptr\n"); }

func(0);      // "int" - 의도대로
func(NULL);   // "int" ← ❌ 포인터 버전이 호출되지 않음!
              //      NULL이 0(int)으로 해석됨
```

nullptr 사용

```
// nullptr: 포인터 전용 null 상수 (타입: nullptr_t)
void func(int n) { printf("int\n"); }
void func(int* p) { printf("ptr\n"); }

func(nullptr); // "ptr" ✅ 포인터 버전이 정확히 호출됨

// 포인터 초기화
int* p = nullptr; // ✅ C++ 권장
// int* p = NULL; // ⚠️ 동작은 하지만 권장 안 함
// int* p = 0;    // ⚠️ 동작은 하지만 의미 불명확

// null 체크
if (p == nullptr) { /* null임 */ }
if (!p)             { /* 동일한 의미 */ }
```

C++에서는 항상 `nullptr` 을 사용하자.

new / delete — C++ 동적 할당

C++에서는 `malloc / free` 대신 `new / delete` 를 사용한다.

단일 객체

```
// C 스타일
int* p1 = (int*)malloc(sizeof(int));
*p1 = 42;
free(p1);

// C++ 스타일
int* p2 = new int;           // 할당 (초기화 없음)
*p2 = 42;
delete p2;                   // 해제

int* p3 = new int(99);       // 할당 + 초기화
delete p3;

// 클래스 객체 — 생성자/소멸자 자동 호출
struct Point { int x; int y; };
Point* pt = new Point{3, 4};
delete pt;                   // 소멸자 자동 호출
```

배열

```
int n = 5;

// C 스타일
int* arr1 = (int*)malloc(n * sizeof(int));
free(arr1);

// C++ 스타일
int* arr2 = new int[n];       // 배열 할당
for (int i = 0; i < n; i++)
    arr2[i] = i * 10;
delete[] arr2;                // ✅ 반드시 delete[] (대괄호 필수!)

// 초기값 지정
int* arr3 = new int[5]{1,2,3,4,5};
delete[] arr3;

// ❌ 혼용 금지
int* p = new int;
free(p);                      // ❌ undefined behavior
int* arr = (int*)malloc(5 * sizeof(int));
delete[] arr;                 // ❌ undefined behavior
```

new VS malloc 차이

malloc

```
#include <stdlib.h>

// 1. 반환형이 void* - 캐스팅 필요
int* p = (int*)malloc(sizeof(int));

// 2. 실패 시 NULL 반환 → 직접 체크
if (p == NULL) { /* 처리 */ }

// 3. 생성자/소멸자 호출 안 함
struct Foo { Foo() { puts("생성"); } };
Foo* f = (Foo*)malloc(sizeof(Foo));
// 생성자 호출 안 됨!

free(p);
free(f);
```

new

```
// 1. 반환형이 정확한 타입 - 캐스팅 불필요
int* p = new int;

// 2. 실패 시 std::bad_alloc 예외 던짐
// (예외 없이 nullptr 반환하려면 nothrow 사용)
int* p2 = new (std::nothrow) int;
if (p2 == nullptr) { /* 처리 */ }

// 3. 생성자/소멸자 자동 호출 
struct Foo {
    Foo() { puts("생성"); }
    ~Foo() { puts("소멸"); }
};

Foo* f = new Foo;    // "생성" 출력
delete f;            // "소멸" 출력

delete p;
```

const 와 포인터

const 의 위치에 따라 의미가 달라진다.

세 가지 조합

```
int a = 10, b = 20;

// 1. 가리키는 값이 const (읽기 전용 포인터)
const int* p1 = &a;
// *p1 = 99;    // ❌ 값 변경 불가
p1 = &b;        // ✅ 다른 변수를 가리키도록 변경 가능

// 2. 포인터 자체가 const (고정 포인터)
int* const p2 = &a;
*p2 = 99;      // ✅ 값 변경 가능
// p2 = &b;    // ❌ 다른 변수를 가리킬 수 없음

// 3. 둘 다 const
const int* const p3 = &a;
// *p3 = 99;   // ❌
// p3 = &b;    // ❌
```

읽는 방법 — 오른쪽에서 왼쪽으로

```
const int* p    // p is a pointer to const int
                // → 가리키는 int값이 const

int* const p    // p is a const pointer to int
                // → p 자체가 const

const int* const p // p is a const pointer to const int
                  // → 둘 다 const
```

함수 인자에서의 활용

```
// 읽기만 할 때 - const int* (또는 const int&)
void print(const int* arr, int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

// 수정할 때 - int*
void fill(int* arr, int n, int val) {
    for (int i = 0; i < n; i++)
        arr[i] = val;
}
```

포인터와 클래스

new 로 객체 생성

```
class Student {
public:
    int age;
    char name[32];

    Student(int a, const char* n) : age(a) {
        strcpy(name, n);
    }
    void greet() { printf("안녕, %s!\n", name); }
};

int main() {
    // 스택 할당 - 스코프 벗어나면 자동 소멸
    Student s1(20, "홍길동");
    s1.greet();

    // 힙 할당 - 직접 delete 필요
    Student* s2 = new Student(21, "이순신");
    s2->greet();    // 포인터는 -> 연산자
    delete s2;      // 소멸자 자동 호출
}
```

객체 배열

```
// 스택 배열
Student arr[3] = {
    {20, "홍길동"},
    {21, "이순신"},
    {22, "강감찬"},
};

// 힙 배열 (기본 생성자 필요)
Student* heap_arr = new Student[3];
heap_arr[0].age = 20;
// heap_arr[0]->age ← ❌ 배열 원소는 . 사용
delete[] heap_arr;    // ✅ delete[] 로 해제

// 포인터 배열 (각각 독립적으로 할당)
Student* ptrs[3];
ptrs[0] = new Student(20, "홍길동");
ptrs[1] = new Student(21, "이순신");
for (int i = 0; i < 2; i++)
    delete ptrs[i];
```

스마트 포인터 (Smart Pointer)

`new` / `delete` 를 직접 쓰면 실수하기 쉽다. C++11부터는 **스마트 포인터**로 메모리를 자동 관리한다.

`unique_ptr` — 단독 소유

```
#include <memory>

// new 없이 make_unique 사용 (C++14)
auto p = std::make_unique<int>(42);

printf("%d\n", *p);    // 42 - 역참조
// delete 불필요 - 스코프 벗어나면 자동 해제

// 클래스 객체
auto s = std::make_unique<Student>(20, "홍길동");
s->greet();            // -> 연산자 동일

// 배열
auto arr = std::make_unique<int[]>(5);
arr[0] = 10;
// delete[] 불필요 - 자동 해제

// 소유권 이전 (복사 불가, 이동만 가능)
auto p2 = std::move(p); // p → p2 로 소유권 이전
// p는 이제 nullptr
```

`shared_ptr` — 공유 소유

```
#include <memory>

// 여러 포인터가 같은 객체를 공유
auto a = std::make_shared<int>(99);
auto b = a;    // 복사 가능 - 참조 횟수 증가 (2)

printf("%d\n", *a);    // 99
printf("%d\n", *b);    // 99
printf("%ld\n", a.use_count()); // 2

b = nullptr; // 참조 횟수 감소 (1)
// a가 소멸될 때 비로소 메모리 해제
```

raw 포인터와 비교

	<code>new</code> / <code>delete</code>	<code>unique_ptr</code>	<code>shared_ptr</code>
소유자	프로그래머	단독	공유
해제	수동	자동	자동
복사	가능	불가 (이동만)	가능

C vs C++ 포인터 비교 요약

항목	C	C++
null 포인터	<code>NULL</code> (= 정수 0)	<code>nullptr</code> (타입 안전)
동적 할당	<code>malloc(size)</code> + 캐스팅	<code>new T</code> (타입 추론, 캐스팅 불필요)
배열 할당	<code>malloc(n * sizeof(T))</code>	<code>new T[n]</code>
해제	<code>free(p)</code>	<code>delete p</code> / <code>delete[] p</code>
실패 처리	<code>NULL</code> 반환 체크	<code>bad_alloc</code> 예외 (또는 <code>nothrow</code>)
생성자/소멸자	호출 안 됨	자동 호출
자동 메모리 관리	없음	스마트 포인터 (<code>unique_ptr</code> , <code>shared_ptr</code>)
참조 타입	없음	<code>int&</code> — 포인터의 안전한 대안

```
// C 스타일 — 여전히 C++에서 동작하지만 권장하지 않음
int* p = (int*)malloc(sizeof(int));
free(p);
```

```
// C++ 권장 스타일
auto p = std::make_unique<int>(0); // 해제 불필요
```

C++ 프로그래밍

스마트 포인터 (Smart Pointer)

왜 스마트 포인터가 필요한가?

raw 포인터는 강력하지만, 직접 관리하면 실수하기 쉽다.

```
1 void process(bool error) {
2     int* data = new int[1000];
3
4     if (error) {
5         return;          // ✖ 메모리 누수! delete[] 없이 반환
6     }
7
8     // ... 작업 ...
9
10    delete[] data;        // 정상 경로에서만 해제됨
11 }
```

raw 포인터의 세 가지 위험:

문제	설명
메모리 누수	<code>delete</code> 없이 함수 반환, 예외 발생 시 해제 코드 미실행
뎅글링 포인터	<code>delete</code> 후 포인터를 계속 사용
이중 해제	같은 포인터를 두 번 <code>delete</code> → 크래시

스마트 포인터는 객체가 소멸될 때 `delete` 를 자동으로 호출해 이 문제를 해결한다. `#include <memory>` 하나로 사용 가능하다.

RAII — 스마트 포인터의 원리

RAII (Resource Acquisition Is Initialization): 자원 획득을 초기화에 묶고, 소멸자에서 자동 해제하는 C++ 관용구다.

직접 구현해보면

```
// 스마트 포인터의 원리를 단순하게 구현
template <typename T>
class SimplePtr {
    T* raw;
public:
    explicit SimplePtr(T* p) : raw(p) {}

    // 소멸자에서 자동 해제
    ~SimplePtr() { delete raw; }

    // 포인터처럼 사용할 수 있도록
    T& operator*() { return *raw; }
    T* operator->() { return raw; }
};

void use() {
    SimplePtr<int> p(new int(42));
    printf("%d\n", *p);
    // 함수 종료 시 ~SimplePtr() 자동 호출 → delete
}
```

표준 스마트 포인터 세 종류

타입	소유 방식	주 용도
<code>unique_ptr<T></code>	단독 소유	일반적인 동적 할당 대체
<code>shared_ptr<T></code>	공유 소유 (참조 카운팅)	여러 곳에서 공유
<code>weak_ptr<T></code>	비소유 관찰	순환 참조 방지

세 종류 중 `unique_ptr` 이 기본 선택이다. 공유가 필요할 때만 `shared_ptr` 을 쓰고, 순환 참조가 생길 때만 `weak_ptr` 로 뚫는다.

unique_ptr — 단독 소유

하나의 unique_ptr 만이 객체를 소유한다. 복사 불가, 이동만 가능하다.

생성과 기본 사용

```
#include <memory>

// ✅ make_unique 사용 권장 (C++14)
auto p = std::make_unique<int>(42);
printf("%d\n", *p);    // 42 - 역참조

// 클래스 객체
struct Point { int x, y; };
auto pt = std::make_unique<Point>(Point{3, 4});
printf("%d\n", pt->x);  // 3 - 멤버 접근

// 배열
auto arr = std::make_unique<int[]>(5);
arr[0] = 10;
arr[1] = 20;
// delete[] 불필요 - 스코프 종료 시 자동 해제
```

소유권 확인

```
auto p = std::make_unique<int>(10);
printf("%p\n", p.get());    // raw 포인터 얻기

if (p) { /* p가 nullptr이 아니면 */ }
p.reset();                  // 명시적 해제, p = nullptr
```

복사 불가 — 이동(move)만 가능

```
auto a = std::make_unique<int>(100);

// auto b = a;                // ❌ 복사 불가 - 컴파일 오류

// ✅ 소유권 이전 (move)
auto b = std::move(a);
// 이후 a는 nullptr, b가 소유자
printf("%d\n", *b);           // 100
if (a == nullptr) puts("a는 이제 비어 있음");

// 함수로 소유권 이전
void consume(std::unique_ptr<int> p) {
    printf("%d\n", *p);
} // 함수 종료 시 p 소멸 → 자동 해제

consume(std::move(b));
// b도 이제 nullptr
```

unique_ptr — 활용 패턴

팩토리 함수

```
struct Animal {
    virtual void speak() = 0;
    virtual ~Animal() = default;
};
struct Dog : Animal {
    void speak() override { puts("멍멍"); }
};
struct Cat : Animal {
    void speak() override { puts("야옹"); }
};

// unique_ptr 반환 - 소유권을 호출자에게 넘김
std::unique_ptr<Animal> make_animal(bool dog) {
    if (dog) return std::make_unique<Dog>();
    else     return std::make_unique<Cat>();
}

int main() {
    auto a = make_animal(true);
    a->speak();    // "멍멍"
}    // a 소멸 → Dog 자동 해제
```

컨테이너에 담기

```
#include <memory>
#include <vector>

std::vector<std::unique_ptr<Animal>> zoo;

zoo.push_back(std::make_unique<Dog>());
zoo.push_back(std::make_unique<Cat>());
zoo.push_back(std::make_unique<Dog>());

for (const auto& animal : zoo) {
    animal->speak();
    // "멍멍", "야옹", "멍멍"
}

// vector 소멸 시 모든 Animal 자동 해제
```

unique_ptr 를 컨테이너에 넣으면 다형성 객체를 안전하게 관리할 수 있다.

shared_ptr — 공유 소유

여러 shared_ptr 이 같은 객체를 공유한다. 참조 카운트가 0이 될 때 자동 해제된다.

생성과 공유

```
#include <memory>

// ✅ make_shared 사용 권장
auto a = std::make_shared<int>(99);
printf("카운트: %ld\n", a.use_count()); // 1

{
    auto b = a;    // 복사 가능 - 카운트 증가
    printf("카운트: %ld\n", a.use_count()); // 2

    auto c = a;    // 또 복사
    printf("카운트: %ld\n", a.use_count()); // 3

    *b = 200;
    printf("%d\n", *a); // 200 - 같은 객체!
} // b, c 소멸 → 카운트 감소

printf("카운트: %ld\n", a.use_count()); // 1
// a 소멸 시 카운트 0 → 비로소 메모리 해제
```

함수 간 공유

```
void display(std::shared_ptr<int> p) {
    // 복사로 전달 - 카운트 증가
    printf("값: %d, 카운트: %ld\n",
           *p, p.use_count());
}

int main() {
    auto p = std::make_shared<int>(42);
    printf("카운트: %ld\n", p.use_count()); // 1

    display(p);
    // 함수 내부: 값: 42, 카운트: 2
    // 함수 반환 후: 카운트 다시 1

    printf("카운트: %ld\n", p.use_count()); // 1
}
```

shared_ptr 은 참조 카운팅 비용이 있다. 공유가 필요 없다면 unique_ptr 이 더 효율적이다.

shared_ptr — 순환 참조 문제

두 객체가 서로를 shared_ptr 로 가리키면 카운트가 절대 0이 되지 않아 메모리가 해제되지 않는다.

문제 상황

```
struct Node {
    int val;
    std::shared_ptr<Node> next;    // 다음 노드
    std::shared_ptr<Node> prev;    // 이전 노드 ← 문제!

    ~Node() { printf("Node %d 소멸\n", val); }
};

int main() {
    auto a = std::make_shared<Node>(1);
    auto b = std::make_shared<Node>(2);

    a->next = b;    // a → b (b 카운트: 2)
    b->prev = a;    // b → a (a 카운트: 2)
}
// main 종료: a, b 지역변수 소멸 → 카운트 각 1
// ❗ 카운트가 1로 남아 소멸자 절대 호출 안 됨!
// → 메모리 누수
```

weak_ptr 로 해결

```
struct Node {
    int val;
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> prev;    // ✅ weak_ptr로 변경

    ~Node() { printf("Node %d 소멸\n", val); }
};

int main() {
    auto a = std::make_shared<Node>(1);
    auto b = std::make_shared<Node>(2);

    a->next = b;    // a → b (b 카운트: 2)
    b->prev = a;    // b → a (a 카운트: 1, weak은 불포함)
}
// main 종료: a 카운트 0 → "Node 1 소멸"
//                b 카운트 0 → "Node 2 소멸" ✅
```


weak_ptr — 비소유 관찰

weak_ptr 은 객체를 소유하지 않고 관찰만 한다. 참조 카운트에 영향을 주지 않는다.

생성과 접근

```
auto sp = std::make_shared<int>(42);
std::weak_ptr<int> wp = sp;    // weak_ptr 생성

printf("카운트: %ld\n", sp.use_count()); // 1
                                // weak은 카운트 불포함

// weak_ptr로 직접 접근 불가 - lock() 필요
// *wp = 10;    // ❌ 컴파일 오류

// lock(): shared_ptr 임시 생성 (원본이 살아있으면)
if (auto locked = wp.lock()) {
    printf("값: %d\n", *locked); // 42
} else {
    puts("객체가 이미 소멸됨");
}
```

객체 소멸 후 접근

```
std::weak_ptr<int> wp;

{
    auto sp = std::make_shared<int>(99);
    wp = sp;

    if (auto lk = wp.lock()) {
        printf("살아있음: %d\n", *lk); // 99
    }
} // sp 소멸 → 카운트 0 → 메모리 해제

// 이미 소멸된 객체에 접근 시도
if (auto lk = wp.lock()) {
    printf("살아있음\n");
} else {
    puts("이미 소멸됨"); // ← 이쪽 실행
}

// expired()로 소멸 여부 확인
printf("만료됨: %d\n", wp.expired()); // 1 (true)
```

커스텀 삭제자 (Custom Deleter)

`delete` 대신 원하는 해제 로직을 지정할 수 있다.

unique_ptr 커스텀 삭제자

```
#include <cstdio>
#include <memory>

// FILE* 를 스마트 포인터로 관리
auto file_deleter = [](FILE* f) {
    if (f) {
        fclose(f);
        puts("파일 닫힘");
    }
};

// 두 번째 타입 인자에 삭제자 타입 지정
std::unique_ptr<FILE, decltype(file_deleter)>
    fp(fopen("test.txt", "w"), file_deleter);

if (fp) {
    fprintf(fp.get(), "Hello, file!\n");
}
// 스코프 종료 시 file_deleter 자동 호출 → fclose
```

shared_ptr 커스텀 삭제자

```
// shared_ptr은 타입 인자 없이 삭제자 전달
auto sp = std::shared_ptr<FILE>(
    fopen("test.txt", "r"),
    [](FILE* f) {
        if (f) fclose(f);
    }
);

// C 라이브러리 핸들 관리 패턴
auto buf = std::shared_ptr<uint8_t>(
    (uint8_t*)malloc(1024),
    [](uint8_t* p) { free(p); }
);

// 여러 곳에서 공유해도 마지막에 free 한 번만 호출
auto buf2 = buf;
```

C 라이브러리의 `FILE*`, `malloc` 메모리 등 `delete` 로 해제할 수 없는 자원도 안전하게 관리할 수 있다.

make_unique VS make_shared VS new 직접 사용

new 직접 사용의 문제

```
// 함수 인자 평가 순서가 비결정적 → 잠재적 누수
void process(std::shared_ptr<int> p, int n);

process(
    std::shared_ptr<int>(new int(42)), // A
    compute()                        // B
);
// A의 new와 shared_ptr 생성 사이에 B가 실행되고
// B에서 예외가 나면 new된 메모리가 누수될 수 있음

// ✅ make_shared/make_unique는 이 문제 없음
process(std::make_shared<int>(42), compute());
```

make_shared의 메모리 효율

```
// new 직접 사용: 할당이 두 번 발생
// 1. int 객체용 힙 메모리
// 2. 참조 카운트 블록용 힙 메모리
std::shared_ptr<int> p1(new int(42));

// make_shared: 할당이 한 번만 발생 ✅
// 객체 + 참조 카운트 블록을 한 번에 할당
auto p2 = std::make_shared<int>(42);
```

권장 사용법 정리

```
// 항상 make 함수를 사용한다
auto up = std::make_unique<T>(args...); // C++14
auto sp = std::make_shared<T>(args...);

// new를 직접 써야 하는 경우
// - 커스텀 삭제자가 필요할 때 (shared_ptr만 해당)
std::shared_ptr<T> p(new T, my_deleter);
```

스마트 포인터 선택 가이드

```
1  동적 할당이 필요한가?
2
3  └─ 단독 소유 (기본 선택)
4      → unique_ptr<T>
5          make_unique<T>(args...)
6
7  └─ 여러 곳에서 공유해야 하는가?
8      → shared_ptr<T>
9          make_shared<T>(args...)
10
11      shared_ptr를 쓰는데 순환 참조가 생기는가?
12          → 한쪽을 weak_ptr<T>로 변경
13
14  └─ 이미 존재하는 C 자원 (FILE*, malloc 등)?
15      → unique_ptr / shared_ptr + 커스텀 삭제자
```

상황	선택
함수 내 지역 동적 할당	<code>unique_ptr</code>
팩토리 함수 반환값	<code>unique_ptr</code>
컨테이너에 다형성 객체 저장	<code>vector<unique_ptr<Base>></code>
캐시, 공유 설정 객체	<code>shared_ptr</code>

요약

```
#include <memory>

// unique_ptr - 단독 소유, 복사 불가, 이동 가능
auto up = std::make_unique<T>(args...);
auto up2 = std::move(up);    // 소유권 이전

// shared_ptr - 공유 소유, 참조 카운팅
auto sp = std::make_shared<T>(args...);
auto sp2 = sp;               // 복사 가능, 카운트 증가
sp.use_count();              // 현재 참조 카운트

// weak_ptr - 비소유 관찰, 순환 참조 방지
std::weak_ptr<T> wp = sp;
if (auto lk = wp.lock()) {   // 살아있을 때만 접근
    // lk 사용
}
wp.expired();                // 소멸 여부 확인
```

타입	복사	이동	참조 카운트	null 가능	주 용도
unique_ptr	✗	✓	없음	✓	일반 동적 할당
shared_ptr	✓	✓	있음	✓	공유 소유
weak_ptr	✓	✓	불포함	✓	순환 참조 방지

규칙: raw new / delete 는 커스텀 삭제자가 필요한 경우 등 특수한 상황에서만 쓰고, 그 외 모든 동적 할당은 스마트 포인터로 대체하라.